

Backend Requirements Addendum

Volunteer Organization Platform — Frontend → Laravel API Handoff

Report date: August 19, 2026 · Revision 2 (incorporates an independent second-pass audit)

Reference document: `docs/Backend-Requirements-Document.pdf` — created August 9, 2026, 2:41 PM. That document remains the fixed baseline and is not reproduced here.

Revision history: Revision 1 (Aug 19, 2026) compared the reference document against the code under `src/services/`, `hooks/`, and `constants/`. Revision 2 (this document) is an **independent audit** of Revision 1 that: (a) diffed the exact pre-reference-document commit (`79bbb84`, Aug 8 2026 22:14) against HEAD across **every changed file in `src/` without exception** — 178 files, not just the services/hooks/constants sampled the first time; (b) grepped the full diff for language patterns that state backend-internal behavior as fact ("الباك اند" الحقيقي, "already exists", "assumed", "confirmed", specific algorithm/class names, etc.); (c) re-verified every one of the reference document's §23.C "Undetermined" items one by one against current code.

Confidence tags: every item below is marked with where its certainty comes from — see legend.

Scope. This document contains only what changed since the reference document was written. Any requirement, field, endpoint, or rule from `Backend-Requirements-Document.pdf` that is not mentioned below is unchanged and remains authoritative exactly as written there. Sections are numbered to match the reference document's own §-numbering; sections with no changes are omitted. A prior, incrementally-built delta file (`docs/Backend-Requirements-Addendum.pdf`) existed earlier in the repository's history but was removed from the working tree before Revision 1 was produced; this document is that file's direct successor.

Legend

Change-type tag	Meaning
NEW	A feature/field/route/endpoint that did not exist in the reference document and now does.
REMOVED	Something the reference document documented that has actually been removed from the code.
CHANGED	Same feature, but its shape/behavior/required fields changed.
RESOLVED	An ambiguity or open question in the reference document that the code now answers definitively — a genuine, observable behavior change, not an assumption.

Confidence-source tag	Meaning
RECONFIRMED	Was already in Revision 1; this audit independently re-derived it from the code a second time and it still holds.
NEW — FROM AUDIT	Discovered only during this second-pass audit; did not appear in Revision 1 at all.

NEEDS HUMAN CONFIRMATION

A claim about actual backend behavior (a PHP method signature, an internal algorithm, a product decision) that cannot be verified by reading frontend code alone. Presented as an open question — not resolved, not guessed, and no side is picked.

§3.4 — Opportunity entity

RECONFIRMED**NEW**

New field: `registrationClosedReason`

Alongside the already-documented `registrationClosedManually` boolean, opportunities now carry a reason discriminator with two possible values: `'organization'` (org manually closed registration via the toggle) and `'city_deactivated'` (see §7.3 below — set automatically when an admin deactivates the opportunity's governorate). Cleared to `null` whenever registration is reopened, regardless of the previous reason.

`src/services/opportunities.js:663-671`

Backend action: add a nullable `registration_closed_reason` column (enum/string: `organization` | `city_deactivated` | `null`) to the `opportunities` table and include it in the Opportunity resource.

§3.8 — Governorate/City entity

RECONFIRMED**NEW**

New field: `isActive`, plus admin enable/disable endpoint

Governorates now carry a boolean `isActive` flag. Admin city management calls `PATCH /governorates/{id}/status { isActive }` to disable/re-enable a governorate instead of deleting it. Inactive governorates stay in the full list (old records still reference them by `nameEn`) but are excluded from selection dropdowns on Register/profile/opportunity-creation forms via a new `getActiveGovernorates()` helper.

`src/services/syrianGovernorates.js:15,19-33,68-69,193-236`

Backend action: add a boolean `is_active` (default true) column to the `governorates` table, and implement `PATCH /governorates/{id}/status`. See §7.3 for the cascading side-effect this endpoint must also trigger.

RECONFIRMED**REMOVED****DELETE /governorates/{id} no longer used by the frontend**

The reference document's §19 catalog listed a full GET/POST/PUT/DELETE set for `/governorates`. The frontend has replaced hard deletion with the soft-disable pattern above:

`src/hooks/queries/useDeleteCityMutation.js` was deleted and

`src/hooks/queries/useToggleCityStatusMutation.js` added in its place (commit `bdd2770`, Aug 19 2026). The code comment is explicit that no delete endpoint exists or is expected: "هون `DELETE` endpoint لا يوجد أصلاً".

`src/services/syrianGovernorates.js:196-201` · git commit `bdd2770`

Backend action: a hard-delete governorate endpoint is no longer required. Do not build **DELETE** `/governorates/{id}` unless product explicitly asks for it again.

RECONFIRMED**RESOLVED****nameAr is now collected by the admin city form**

The reference document flagged (§23.B) that `CityFormModal` only exposed a `nameEn` input even though the data model requires `nameAr` too, with "no way for an admin to supply it." `CityFormModal.jsx` now has two required inputs — Arabic name and English name — and both are sent on create/update. `nameEn` becomes read-only once a city is edited (existing records reference it by value; renaming it would orphan them), while `nameAr` stays editable at any time. This is a genuine, observable behavior change — not an assumption — so it stands as fully RESOLVED.

`src/components/admin/CityFormModal.jsx:49-81`

§4 — User Roles & Permissions

RECONFIRMED**RESOLVED****Admin role-priority bug fixed to match the already-documented order**

The reference document already specified role-resolution priority as admin > organization > volunteer (§4). The frontend's `resolveAccountType()` previously checked organization/volunteer before admin; per the code's own comment, this meant "ما في أدمن قادر يسجل دخول أساساً" — no admin account could log in at all. This has been fixed to check admin first. No change to the documented contract itself, but worth flagging in case the backend team was working around the old (broken) frontend behavior in any way.

`src/services/auth.js:81-90`

§6 — Route Map

RECONFIRMED **NEW** Four new routes added since the reference document

Path	Access	Page
<code>/notifications</code>	Volunteer, Organization, Admin (shared)	Full notification history/list (previously only the Navbar bell existed)
<code>/my-journey</code>	Volunteer only	Achievements + completed-opportunities timeline (replaces the old standalone achievements view; <code>/achievements</code> now legacy-redirects here)
<code>/admin/volunteers</code>	Admin only	Read-only volunteer roster (see §12)
<code>/admin/opportunities</code>	Admin only	Read-only platform-wide opportunity list (see §12)

`src/App.jsx:36,40-41,72-80,85,104-105 · src/constants/paths.js:13,21,26-27,41`

§7.3 — Opportunity Status Transitions

RECONFIRMED **NEW** New system-triggered transition: governorate deactivation auto-closes registration

Not present in the reference document's transition table. When an admin deactivates a governorate (§3.8), every opportunity located in that governorate that is currently `registration_open` is automatically flipped to registration-closed, with `registrationClosedReason: 'city_deactivated'`. Opportunities that are already `in_progress`, `completed`, or closed for another reason are left untouched. Re-activating the governorate later does **not** automatically reopen these opportunities — deliberate, per the code comment — the organization must reopen manually via the existing toggle.

`src/services/opportunities.js:683-711`

Backend action: when handling `PATCH /governorates/{id}/status` with `isActive: false`, the backend must apply the same cascade server-side — bulk-close registration (with reason `city_deactivated`) on every `registration_open` opportunity in that governorate. This is currently only simulated in the frontend's mock layer; real-mode has no server logic to fall back on.

§10 — Volunteer Requirements

RECONFIRMED

CHANGED

Education level enum reduced from 6 to 4 values — reference document's recommendation reversed

The reference document (§10, §20) flagged that the Zod schema allowed 6 education levels while the form only rendered 4, and explicitly recommended: *"backend should accept all 6 regardless, this is a frontend gap, not a backend one."* That gap has since been closed in the opposite direction: **Master's Degree** and **PhD** have been removed from the validation schema entirely (not just hidden in the UI), per an explicit product decision recorded in the code comment: volunteering-platform registrants holding those degrees don't fit the platform's context. The allowed set is now exactly: **No Formal Education, High School, Diploma, Bachelor's Degree**.

src/utils/auth/VolunteerProfileValidation.js:18-27

Backend action: reverse any prior guidance to accept 6 education-level values — the `education_level` validation rule (Laravel form request) should now allow only these 4 values, matching the frontend exactly.

RECONFIRMED

RESOLVED

Two profile-save bugs fixed to match the already-documented contract

- **City validation:** `city` existed in the profile form but was missing from the Zod schema, so it could be saved empty despite being one of the profile-completion-gate's required fields (§5.7). It is now a required `requiredSelect` field, consistent with what the reference document already listed as mandatory. src/utils/auth/VolunteerProfileValidation.js:65-67
- **skill_ids now actually sent:** the real-mode profile-update request never included `skill_ids` in its multipart body at all, so any skill change a volunteer made was silently dropped in real mode (mock mode was unaffected). The field is now appended as `JSON.stringify(values.skills)`, matching the field name the reference document already specified in §3.2/§5.6. src/services/volunteer.js:40-44

Both are bug fixes bringing the implementation in line with the contract the reference document already described — no new backend action required, but useful context if the backend team observed volunteers' skill edits not persisting.

§11 — Organization Requirements & Verification Lifecycle

RECONFIRMED

CHANGED

Organization profile update endpoint: exact multipart contract now defined (was previously plain JSON PUT)

The reference document's §11.3 described `PUT /organizations/{id}` as JSON-only, with the update controller confirmed to only handle plain field updates (no file support). The frontend has since moved this endpoint onto the multipart POST + `_method=PUT` pattern (per the global convention in §2) and now defines the exact fields the backend needs to support:

- `name`, `description`, `city`, `website`, `contact_person` — unchanged text fields
- `photo` (file) — organization logo, sent only when a new file is chosen
- `remove_photo = "1"` — new, sent when the org explicitly removes its logo without uploading a replacement; the previous version of this code did not send this signal at all

Code comments confirm the gap itself is still open server-side: *"الباك اند حاليًا (OrganizationController::update) لسا ما بيعالج hasFile('photo') إطلاقًا"* — the backend still silently ignores the file field today, so uploads are safely no-ops rather than errors, but logos still cannot be changed post-registration.

`src/services/organization.js:104-179`

Backend action: when implementing logo-upload support on `OrganizationController::update` (already tracked in the reference document's §22 checklist), use these exact field names: `photo` for the file and `remove_photo` (truthy string) to clear an existing logo without a replacement.

RECONFIRMED**RESOLVED (frontend only)**

Pending-organization action gating clarified (answers reference doc §23.C open question #8)

The reference document listed as undetermined: "*Whether pending organizations should be allowed to post opportunities at all before verification.*" A new shared hook now codifies the intended behavior explicitly: a pending/rejected/suspended organization can see its entire dashboard and every page, but every actual write action (publish/edit/delete an opportunity, accept/reject an applicant, log hours) is disabled client-side until `status === 'verified'`.

src/hooks/useOrganizationVerification.js:1-49

*Important distinction: this is a real, observable frontend behavior change (RESOLVED on the frontend), **not** the same thing as the underlying backend contract question being settled — the actual API endpoints have no server-side enforcement of this rule yet, so a pending org that calls the API directly (bypassing the UI) is not actually blocked today.*

Backend action: this is currently a client-side-only restriction (button-disable), matching the reference document's original §4 warning that "the frontend guard is UI-only." The backend must independently re-enforce the verified-only gate on every relevant write endpoint (opportunity create/update/delete/status-toggle, participation accept/reject, hours update) — a pending org calling these endpoints directly must be rejected server-side regardless of what the UI allows.

RECONFIRMED**NEW**

Organization profile-completion reminder (frontend-only, no backend impact)

A new `isOrganizationProfileComplete()` check (description + city present) now drives a non-blocking reminder banner on the org dashboard, mirroring the volunteer completion gate's spirit but without redirect-blocking. No new fields or endpoints required — listed here only for completeness since it's a new concept not present in the reference document.

src/utils/auth/profileCompletion.js:31-45

§12 — Admin Requirements

RECONFIRMED **NEW** Two new read-only admin monitoring endpoints expected

Beyond the organization-review endpoints already documented, the admin area now has two new pages (§6) backed by two new service calls, both read-only with no filtering/action support:

Method & Path	Response	Notes
GET <code>/admin/volunteers</code>	Array of volunteers: id, name, email, city, created_at, skills_count, opportunities_joined_count	No route exists yet server-side per the frontend comment
GET <code>/admin/opportunities</code>	Array of opportunities: id, title, organization_name, city, status, created_at, start_date	Same — not yet built

`src/services/admin.js:162-267` · `src/hooks/queries/useAdminVolunteersQuery.js`,
`useAdminOpportunitiesQuery.js`

Backend action: add `GET /admin/volunteers` and `GET /admin/opportunities` to the admin namespace (already tracked as blocking in §22 for the organization endpoints — extend the same `role:admin` middleware to these two). Both are read-only/monitoring only — no create/update/delete actions are expected.

RECONFIRMED **CHANGED** `admin.dashboard` query key — still unused, status unchanged

The reference document flagged this cache key as dead code or a signal of a planned-but-unbuilt endpoint. It is **still defined but unused** by any hook after this round of changes — independently re-confirmed during this audit by searching the codebase for any reader of `queryKeys.admin.dashboard` (none found). No change in status; re-flagging so the backend team doesn't assume it was resolved.

`src/app/queryKeys.js:57`

§13 — Achievements System

NEEDS HUMAN CONFIRMATION

Frontend code asserts the group-activity achievement rule already exists server-side

First raised in Revision 1 of this addendum; this audit independently reconfirms the underlying comment is still present, word-for-word identical, in the same 5 files.

The reference document's §13/§22/§23.C left open whether achievement `a3` ("Completion of Three Group Activities") should specifically require `isGroup = true` opportunities, and listed server-side achievement-granting logic as a still-blocking, unbuilt item. Five separate places in the frontend now carry the identical comment, stated as established fact rather than a TODO:

يغذي منطق إنجاز '3 أنشطة جماعية' الموجود بالفعل بالباك اند"
(`AchievementService::checkThreeGroupActivities`)"
("feeds the '3 group activities' achievement logic that already exists in the backend")

`src/utils/opportunityValidation.js:74-76 · src/components/organization/CauseForm.jsx:405-407 ·`
`src/components/organization/MyCauseCard.jsx:124-126 ·`
`src/pages/opportunities/OpportunityDetailsPage.jsx:305-308 ·`
`src/components/opportunity/OpportunityCard.jsx:89-92`

Contradiction to flag: the frontend's own mock achievement calculator has *not* been updated to match this claim — `computeAchievementsForVolunteer()` in `src/services/achievements.js:80-95` still unlocks `a3` from any 3 completed opportunities, with no `isGroup` filter at all. This claim is a statement about backend PHP source the frontend cannot verify by itself, and it is not adopted as fact here — it is presented strictly as an open question below.

Question for backend team: does `AchievementService::checkThreeGroupActivities` exist, and does it filter on the opportunity's `isGroup` flag specifically? Please answer directly — this document does not assume either answer.

§14 — Notifications

RECONFIRMED

CHANGED

Read-marking endpoint now concretely defined: **POST**, not the **PATCH**-style mechanism the reference document speculated

§14 anticipated "a **PATCH** `/notifications/{id}/read`-style mechanism." The frontend has since committed to a concrete real-mode implementation: `onDismiss: () =>`

`apiClient.post(`/notifications/${item.id}/read`)` — a **POST**, not **PATCH**. The list-fetch call also now sends `?unread=true` defensively (with a client-side fallback filter in case the backend ignores it, mirroring the same defensive pattern already documented for **GET** `/organizations`).

`src/services/notifications.js:289-310`

Backend action: implement **POST** `/notifications/{id}/read` (not **PATCH**) to mark a single notification as read. **GET** `/notifications` should ideally honor an `?unread=true` query param, though the frontend will filter client-side if it's ignored.

RECONFIRMED

NEW

Two new notification types, one of them a new admin-facing notification capability

Type	Audience	Trigger
<code>applicant-new</code>	Organization	A volunteer applies (status=pending) to one of the org's opportunities and the org hasn't seen it yet
<code>pending_organization</code>	Admin	A new organization registers and is awaiting verification, not yet seen by any admin

The reference document's type list (achievement, hours, status-accepted, status-rejected, opportunity-reminder, org-verified, org-rejected, applicant-withdrawn) had no admin-facing notification at all — admins receiving notifications about pending organizations is a new capability, not just a new type value.

`src/services/notifications.js:159-230` · `src/constants/notificationTypes.js:17,29,43`

Minor frontend-internal inconsistency: `pending_organization` (underscore) is inconsistent with every other type value's kebab-case naming (`applicant-new`, `org-verified`, etc.), and it has no entry in `NOTIFICATION_TYPE_ICONS/_LABELS` in `notificationTypes.js`, so it silently falls back to the generic bell icon. Flagging for the frontend team, not a backend action.

§15 — Dashboards & Analytics (Organization)

RECONFIRMED

RESOLVED

recentActivity scope clarified: pending applicants only

The reference document's response shape listed **recentActivity** simply as "top 5 by date desc" without specifying which statuses qualify. The implementation now explicitly filters to `status === 'pending'` before taking the top 5 (filtering happens before the slice, not after), per the code's own comment: recent activity is meant to surface what the org still needs to act on, not a general activity log including already-decided applications.

src/services/dashboard.js:285-305

Backend action: when building the eventual `GET /organizations/me/dashboard` aggregate endpoint (§22), **recentActivity** should be the 5 most recent **pending** participations only, ordered by application date descending — not the 5 most recent participations regardless of status.

§16 — File Uploads (addendum)

NEW — FROM AUDIT

NEEDS HUMAN CONFIRMATION

Two specific claims about volunteer-photo handling in the real Laravel backend, both unverifiable from the frontend

Discovered only during this second-pass audit — not present in Revision 1. Two related, highly specific comments assert internal backend behavior as settled fact:

- **"Confirmed bug" claim:** a comment states `VolunteerResource.php`'s `photo` accessor calls Spatie's `getFirstMedia()` instead of `getFirstMediaUrl()`, so it returns a raw `Media` object instead of a URL string — unlike `OrganizationResource`, which the same comment says does this correctly. The frontend has added a defensive fallback (try `.original_url/.url` on the object, else ignore it) specifically to work around this. `src/utils/auth/normalizeUser.js:7-11`
- **"Already ready" claim:** a separate comment, while explaining that `OrganizationController::update` still doesn't handle file uploads, states this is "يلي جاهز `VolunteerController::update` بعكس" — "unlike `VolunteerController::update`, which is ready" (i.e., already correctly handles multipart file uploads today). `src/services/organization.js:152-154`

Both read as though someone directly inspected the backend PHP source, but a frontend-only audit has no way to confirm either claim independently, and the reference document does not mention either one.

Questions for backend team:

- 1) Does `VolunteerResource.php`'s `photo` field really call `getFirstMedia()` instead of `getFirstMediaUrl()`?
- 2) Does `VolunteerController::update` already correctly handle a multipart `photo` file upload today?

Please answer both directly — neither is assumed true or false here.

§17 — Filtering, Search, Sort, Pagination (addendum)

NEW — FROM AUDIT

NEW

§17.1 — Default sort order for `GET /opportunities`

Discovered only during this audit. Mock-mode `fetchOpportunities()` now sorts results by `registerEndAt` ascending (soonest registration deadline first) by default — the reference document's §17.1 said nothing about ordering. A TODO comment on the real-mode branch flags that this ordering is **not guaranteed** from the live endpoint and that an explicit `sort` param may need to be added once the endpoint is built.

`src/services/opportunities.js:224-241`

Backend action: confirm whether `GET /opportunities` will return results ordered by `register_end_at` ascending by default. If not, the frontend will add an explicit `?sort=register_end_at` param — please confirm the expected param name/format so both sides agree in advance.

NEW — FROM AUDIT

NEEDS HUMAN CONFIRMATION

§17.2 — Suggested-opportunities matching algorithm stated as "Naive Bayes," contradicting the reference document's own statement of flexibility

Discovered only during this audit — this is the single most important new finding. The reference document's §17.2 explicitly says: *"Response is loosely typed on purpose... backend has flexibility in the exact field name for its match output"* — i.e., at reference-document time the algorithm was deliberately treated as unknown/flexible. Code comments now assert something much more specific and stated as settled fact:

"(على المهارات والمدينة فقط، خصائص مستقلة احتماليًا عن بعضها Naive Bayes خوارزمية الباى اند الحقيقية)"
("the real backend's algorithm — Naive Bayes, on skills and city only, probabilistically independent features")

...and separately, a "قرار مؤكد" ("confirmed decision"): the algorithm uses only existing profile data, with no interaction/click-tracking signal, and none is planned.

`src/services/opportunities.js:143-150, 259-262`

Contradiction to flag: this directly conflicts with the reference document's own framing of this endpoint as flexible/unconfirmed. It's also worth noting the frontend's own mock implementation of `computeMatchScore()` is *not* actually a Naive Bayes model — it's a simple weighted sum (`matchingSkills × 3 + sameCity × 2`), which does not by itself corroborate the specific algorithm claim.

`src/services/opportunities.js:159-183`

Questions for backend team:

- 1) Is `GET /volunteers/me/suggested-opportunities` actually implemented with Naive Bayes on skills + city only?
 - 2) Is it correct that no interaction/click-tracking signal exists or is planned for this algorithm?
- Please confirm or correct directly — this document takes no position on which is true.

NEW — FROM AUDIT

NEEDS HUMAN CONFIRMATION

§17.4 — Should organizations see a volunteer applicant's email address?

Discovered only during this audit. The mock volunteer-profile fixtures used by the applicants list now include an `email` field, justified by a comment stating that the real `VolunteerResource` always includes `user.email`, so mock data "must match this reality." That part (email existing on the base User resource) is consistent with the reference document's §3.1. What the reference document never addresses is the separate, product-level question of whether that email should be *exposed to an organization* on the applicant card/list — that's not something derivable from either the code or the reference document.

`src/services/mock/mockParticipationsStore.js:72-76`

Question for backend/product team: should the applicant-facing volunteer resource (used by `GET /opportunities/{id}/participants`) include the volunteer's email address for the organization to see? If not, it should be explicitly excluded from that specific resource shape.

NEW — FROM AUDIT

RESOLVED

§17.3 — Organizations directory pagination UI now built (answers reference doc §23.C open question #9)

Discovered only during this audit — missed in Revision 1 entirely. The reference document's §23.C question #9 was: "*Whether `GET /organizations` pagination (`paginate(15)`) needs frontend-side 'load more'/page controls to be added — no such UI was found, meaning only the first page may currently ever be visible.*" That gap is closed: `useOrganizationsQuery` now uses React Query's `useInfiniteQuery` with real page params, and `fetchOrganizations()` sends `?page=` and reads `meta.current_page/meta.last_page` to drive it, in both mock and real branches. This is a genuine, observable behavior change — not an assumption — so it is marked fully RESOLVED, unlike the items above.

`src/hooks/queries/useOrganizationsQuery.js:1-40 · src/services/organizations.js:227-306`

Also newly clarified in the same code: the response is double-wrapped — the outer Laravel API Resource envelope (`{data, meta?, links?}`) is unwrapped by `client.js`, but the inner payload is itself a full Laravel Paginator shape (`{data, links, meta}`). So `meta/links` must be read from `response.data.meta/.links`, not from the top-level `response.meta/.links` the outer unwrap would otherwise produce. `src/services/organizations.js:277-284`

§20 — Validation Rules Reference

RECONFIRMED **NEW** Two new opportunity date/hours constraints

Rule	Value	Source
Opportunity start date lower bound	<code>startDate</code> cannot be earlier than the opportunity's creation date	<code>opportunityValidation.js:116-125</code>
Registration window lower bound	<code>registerStartAt</code> cannot be earlier than the opportunity's creation date	<code>opportunityValidation.js:127-133</code>
Total vs. per-volunteer hours	<code>totalHours</code> $\geq$ <code>maxHours</code> (a full opportunity budget must be able to cover at least one volunteer committed at the per-volunteer maximum)	<code>opportunityValidation.js:89-92</code>

Backend action: add matching server-side validation on opportunity create/update: reject a `start_date` or `register_start_at` earlier than "now" (creation time), and reject `total_hours` $<$ `max_hours`.

RECONFIRMED **CHANGED** Education level: 4 allowed values, not 6

Cross-reference of §10 above — the validation-rules table's "Education level: one of 6 fixed values" entry is now stale. See §10 for the full explanation and the new list of exactly 4 values.

§21 — Business Rules the Backend Must Enforce (addendum)

- RECONFIRMED** **12.** Deactivating a governorate must cascade server-side: every `registration_open` opportunity in that governorate is automatically closed for registration (reason: city deactivated), and reactivating the governorate must *not* automatically reopen them (see §7.3).
- RECONFIRMED** **13.** Organization write actions (create/edit/delete/toggle-status opportunities; accept/reject participations; log hours) must be blocked server-side for any organization whose status is not `verified` — the frontend now only disables the buttons, it does not prevent a direct API call (see §11).

§22 — Backend Team Checklist (addendum — new items only)

This supplements, not replaces, the reference document's own §22 checklist — everything listed there that is not mentioned below is still outstanding exactly as described. Items marked Δ are contingent on an answer in the final "Items Requiring Direct Backend Team Confirmation" section below — do not build against them until confirmed.

- Add `registration_closed_reason` nullable column to `opportunities` (values: `organization` | `city_deactivated`).
- Add `is_active` boolean column to `governorates`, default `true`.
- Build `PATCH /governorates/{id}/status { isActive }`, including the auto-close cascade described in §7.3/§21. No delete endpoint is needed for governorates.

- Build `GET /admin/volunteers` and `GET /admin/opportunities` (read-only, `role:admin`) alongside the already-tracked organization-review admin routes.
- Implement `POST /notifications/{id}/read` (confirmed as POST, not PATCH).
- When building logo-upload support on `OrganizationController::update`, accept a `photo` file field and a `remove_photo` field.
- Update the `education_level` validation rule to accept only 4 values (No Formal Education / High School / Diploma / Bachelor's Degree) — Master's and PhD are intentionally excluded now, reversing the reference document's earlier guidance to accept all 6.
- Server-side re-enforce the "verified organizations only" restriction on every opportunity/participation/hours write endpoint — currently UI-only (§11, §21).
- Add matching validation for the two new opportunity rules in §20 (creation-date lower bound on dates; `total_hours >= max_hours`).
- Confirm the default/expected sort order for `GET /opportunities` (§17.1).
- [△](#) Confirm whether `AchievementService::checkThreeGroupActivities` already exists and uses `isGroup` — see final confirmation section.
- [△](#) Confirm the suggested-opportunities matching algorithm and whether `VolunteerResource/VolunteerController` claims are accurate — see final confirmation section.

§23 — Frontend ↔ Backend Contract Gaps (addendum)

B. Frontend-internal inconsistencies — status update

Inconsistency (reference doc)	Current status
CityFormModal only exposes <code>nameEn</code>	RECONFIRMED RESOLVED — now collects both <code>nameAr</code> and <code>nameEn</code> (§3.8)
Gender: schema allows 2, GenderBadge supports a 3rd ("Prefer not to say")	RECONFIRMED unchanged — still open. <code>src/components/common/GenderBadge.jsx:4-8</code> still defines the third option; <code>VolunteerProfileValidation.js:16</code> still only allows Female/Male.

C. Undetermined — status update (all 9 reference-document items re-verified individually)

#	Reference document question	Status after this audit
1	Token expiry & refresh strategy	RECONFIRMED unchanged — no refresh/expiry logic anywhere in <code>api/client.js</code> or <code>auth.js</code> .
2	Server-side logout/token revocation	RECONFIRMED unchanged — <code>AuthContext.logout()</code> is still pure client-side storage clear, no API call. <code>src/context/AuthContext.jsx:79-85</code>
3	Re-applying after rejected/withdrawn: new row or reset?	NEW — FROM AUDIT — see detail below. Not resolved , only a frontend assumption.

4	<code>citiesCoveredCount</code> on platform stats	RECONFIRMED unchanged — comment still asks the backend team to confirm or drop it. <code>src/services/stats.js:5-7</code>
5	Achievements platform-wide vs. per-organization	RECONFIRMED unchanged — still computed across the whole platform, no org scoping.
6	Should <code>a3</code> require <code>isGroup</code> ?	See §13 — NEEDS HUMAN CONFIRMATION , not resolved.
7	Opportunity cover-image field/collection name	RECONFIRMED unchanged — still field <code>image</code> , still no <code>MEDIA_COLLECTIONS</code> entry (commented out placeholder). <code>src/constants/media.js:13-14</code>
8	Should pending orgs be allowed to post opportunities?	See §11 — RECONFIRMED RESOLVED (frontend only) ; backend enforcement still open.
9	<code>GET /organizations</code> pagination UI	See §17.3 — NEW — FROM AUDIT , genuinely RESOLVED .

NEW — FROM AUDIT

NEEDS HUMAN CONFIRMATION

Detail on #3 — the frontend has made an implementation choice, but this does not settle the backend's design question

The mock's `participateInOpportunity()` now clearly creates a **new** participation row when re-applying, while blocking a duplicate only when an *active* (pending/accepted) row already exists for that volunteer+opportunity pair — `withdrawn/rejected` rows no longer block a fresh apply, and are left in place rather than reused.

`src/services/opportunities.js:590-610`

*This is genuinely different from the pagination item above (§17.3): here, the real endpoint is a single **POST** `/opportunities/{id}/participate` call with no parameter that could express "reset vs. new row" — the frontend's mock behavior is a testing convenience, not something the live endpoint contract can currently confirm or deny. The real backend could just as easily reset the existing row to **pending**, and the frontend would behave identically either way from its own perspective (since it just re-fetches the participation list after applying). This must stay an open question, not a resolved one.*

Question for backend team: when a volunteer re-applies after `rejected/withdrawn`, will the real endpoint create a new `opportunity_volunteer` row, or reset the existing one back to **pending**? Either is compatible with the frontend today, but it affects whether a volunteer's full application history for one opportunity is ever visible on either side.

End of addendum. Read together with [Backend-Requirements-Document.pdf](#) (Aug 9, 2026), these two documents describe the complete current state of the frontend's backend requirements as of August 19, 2026. See the final section below for a single consolidated list of every open question raised across this document.

Items Requiring Direct Backend Team Confirmation

Every **NEEDS HUMAN CONFIRMATION** item in this document, gathered in one place. None of these are guessed, resolved, or assumed anywhere above — each needs a direct yes/no/clarification from the backend team before anyone builds or relies on it.

1. **(§13, Achievements)** Does `AchievementService::checkThreeGroupActivities` exist in the real backend, and does it filter specifically on the opportunity's `isGroup` flag? Frontend comments assert yes; the frontend's own mock does not implement this filter.
2. **(§17.2, Suggested Opportunities)** Is the real `/volunteers/me/suggested-opportunities` algorithm actually Naive Bayes based only on skills + city? Is it confirmed that no interaction/click-tracking signal exists or is planned? This directly contradicts the reference document's own statement that the algorithm was left flexible/unconfirmed.
3. **(§16, File Uploads)** Does `VolunteerResource.php`'s `photo` field call Spatie's `getFirstMedia()` instead of `getFirstMediaUrl()` (returning a raw Media object instead of a URL string)? And separately: does `VolunteerController::update` already correctly handle multipart photo-file uploads today?
4. **(§17.4, Applicants list)** Should organizations be able to see a volunteer applicant's email address on the applicant card/list? This is a product/privacy decision, not something derivable from existing code or the reference document.
5. **(§23.C #3, Participation Lifecycle)** When a volunteer re-applies to an opportunity after a prior `rejected` or `withdrawn` participation, does the real backend create a brand-new pivot row, or reset the existing row back to `pending`? The frontend's mock now assumes "new row," but this is a testing convenience, not evidence of the real endpoint's behavior.